Full Deployment Guide

EC2 Web Server with RDS on AWS (Terraform)

Supervised by Eng. Azza Khalil

Team Members:

1. Eslam Farag Mohamed
2. Mina Morcos Ishak
3. Seif Allah Osama Ahmed

Project Overview

This code creates an integrated infrastructure on AWS using Terraform to automatically manage and configure cloud computing resources.

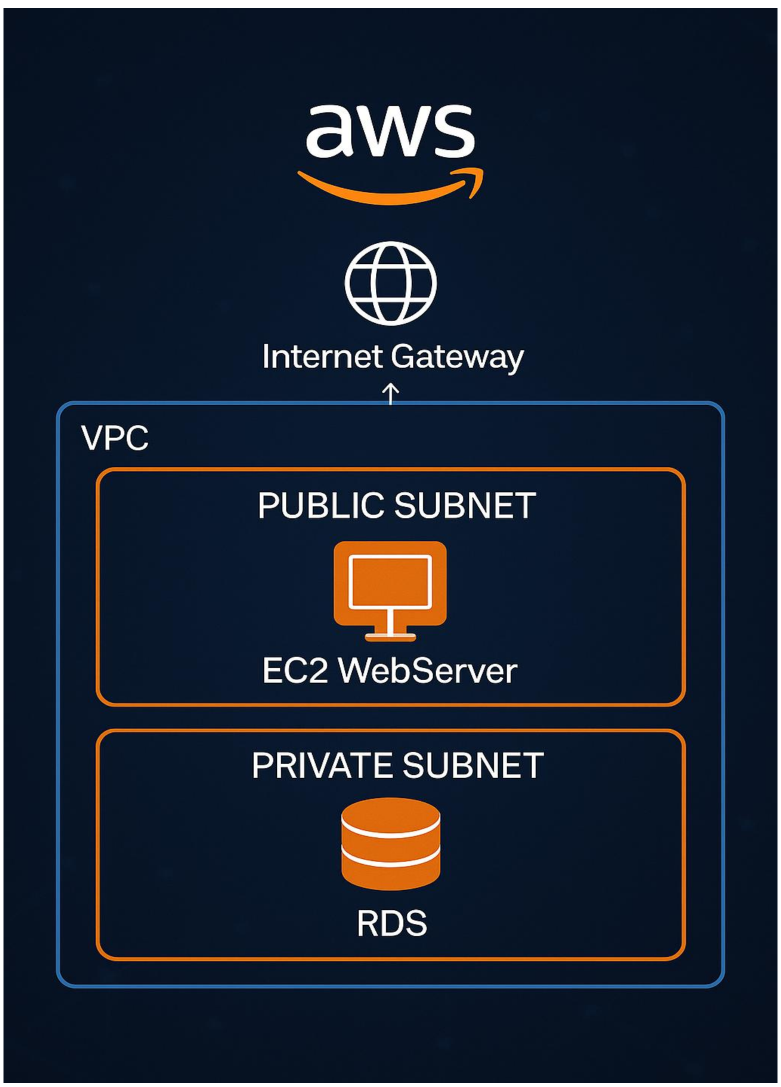
The code includes setting up a virtual private network (VPC) with public and private subnets, creating internet gateways and NAT to securely connect the network to the internet.

The code also creates a MySQL database using RDS, and an EC2 server configured to run a Node.js web application and connect directly to the database.

In addition, an RSA key is generated and an AWS key pair is configured for secure login on the server. S3 is used to store Terraform state files to ensure secure and organized change management.

This code provides a flexible, scalable, and manageable infrastructure without the need for manual intervention, helping accelerate application deployment and deployment.

Architecture Diagram:



Step 1: AWS provider (provider.tf)

Terraform uses the AWS provider (hashicorp/aws) **version 5.36.0.** The AWS provider defines the access keys (**access_key** and **secret_key**) and the **us-east-1** region.

Step 2: networking (vpc.tf)

1-VPC and Subnets

Creates a VPC named `**myvpc**` with the address `**10.0.0.0/16**`, containing a **public subnet** (`10.0.0.0/24`) and a **private subnet** (`10.0.1.0/24`) in two different zones.

Your VPCs (3) Info — Last updated less than a minute ago — Actions — Create VPC

Find VPCs by attribute or tag

Name	VPC ID	State	Block Public...	IPv4 CIDR	IPv6 CIDR
myvpc	vpc-000a534ea73392b93	Available	Off	10.0.0.0/16	–

Subnets (5) Info — Last updated 1 minute ago — Actions — Create subnet

Find subnets by attribute or tag

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
public-subnet	subnet-035beddaa116dd898	Available	vpc-000a534ea73392b93 | myvpc	Off	10.0.0.0/2
private-subnet	subnet-0e5075a1efa5e989d	Available	vpc-000a534ea73392b93 | myvpc	Off	10.0.1.0/2

2-Internet Gateway

Adds an internet gateway connected to the VPC to provide internet connectivity to the public subnet.

Internet gateways (2) Info — Actions — Create internet gateway

Find internet gateways by attribute or tag

Name	Internet gateway ID	State	VPC ID	Owner
igw	igw-09b360d349b9e8bd0	Attached	vpc-000a534ea73392b93 | myvpc	337909736881

3-Route Tables

Sets up a public route table (connects to the internet via an IGW) and a private route table (connects to the internet via a NAT gateway).

Route tables (5) Info — Last updated 4 minutes ago — Actions — Create route table

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
–	rtb-0aeac153722d29aee	–	–	Yes	vpc-0d326bb98a60ca447
private-route-table	rtb-0ad473d886cee637e	subnet-0e5075a1efa5e9...	–	No	vpc-000a534ea73392b93
public-route-table	rtb-0d36bac299f252516	subnet-035beddaa116dd...	–	No	vpc-000a534ea73392b93

4-Route Associations

Associates each subnet with the correct route table: the public subnet to the public route table, and the private subnet to the private route table.

5-Elastic IP and NAT Gateway

Elastic allocates an IP and builds a NAT Gateway within the public subnet to allow the private subnet to exit to the internet.

NAT gateways (2) Info

Actions | Create NAT gateway

Find NAT gateways by attribute or tag

Name	NAT gateway ID	Connectivity...	State	State message	Primary public I...	Primary priva
–	nat-062b95f3816b7fb77	Public	Available	–	13.219.38.129	10.0.0.48

Step 3: Security Groups (security_group.tf)

1-Creates a Security Group for the RDS connection named `**rds-sg**`, and one for EC2 named `**web-sg**` in the same VPC.

2-Temporarily opens all inbound traffic (protocols + ports) to the web server using `-1`.

3-Opens all outbound traffic from the web server to any location.

4- Adds a special rule in RDS that opens port **3306** (MySQL) only for servers in `web-sg`.

5-Adds an explicit rule that opens port **3000** for the Node.js app to receive requests from the internet.

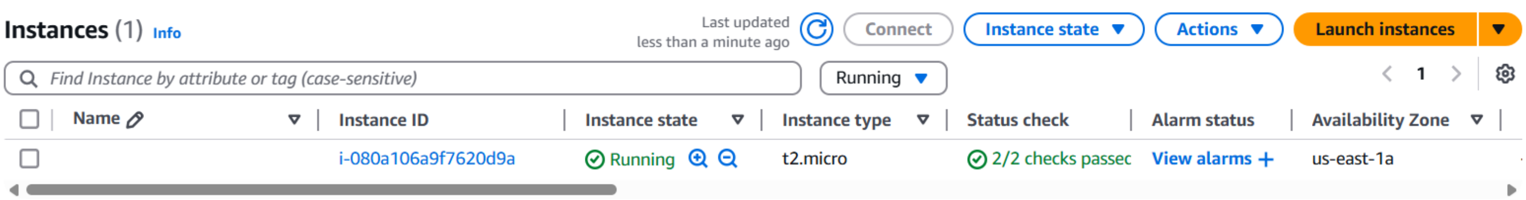
- Only allows EC2 (not public) to access the database.

–	sg-04fc122a4909f1e02	rds sg	vpc-000a534ea73392b93	SG for RDS
–	sg-074c79b9bd24143c3	default	vpc-0e046182b24321f1f	default VPC secur
–	sg-0354afd9ce4abd785	web sg	vpc-000a534ea73392b93	SG for Web Serve

---

Step 4: Launch EC2 Instance (ec2.tf)

- A web server inside the public subnet.
- Replace the AMI with your region-specific **Amazon Linux 2 AMI**.
- Generates a 4096-bit **RSA key** using `tls_private_key` and saves a local copy of the private key in the `**terraform-key.pem**` file.
- Creates an **AWS key pair** using the generated public key and binds it to EC2 to allow SSH connections.
- Creates an EC2 instance on **a public subnet**, bound to the appropriate security group (`**web-sg**`), and with a public IP address for external access.
- Provisioning Within **`remote-exec`**, it sets environment variables, downloads all required software, modifies `index.js` to listen to `0.0.0.0`, and runs the application in the background.



Step 5: Setup RDS MySQL in Private Subnet (rds.tf)

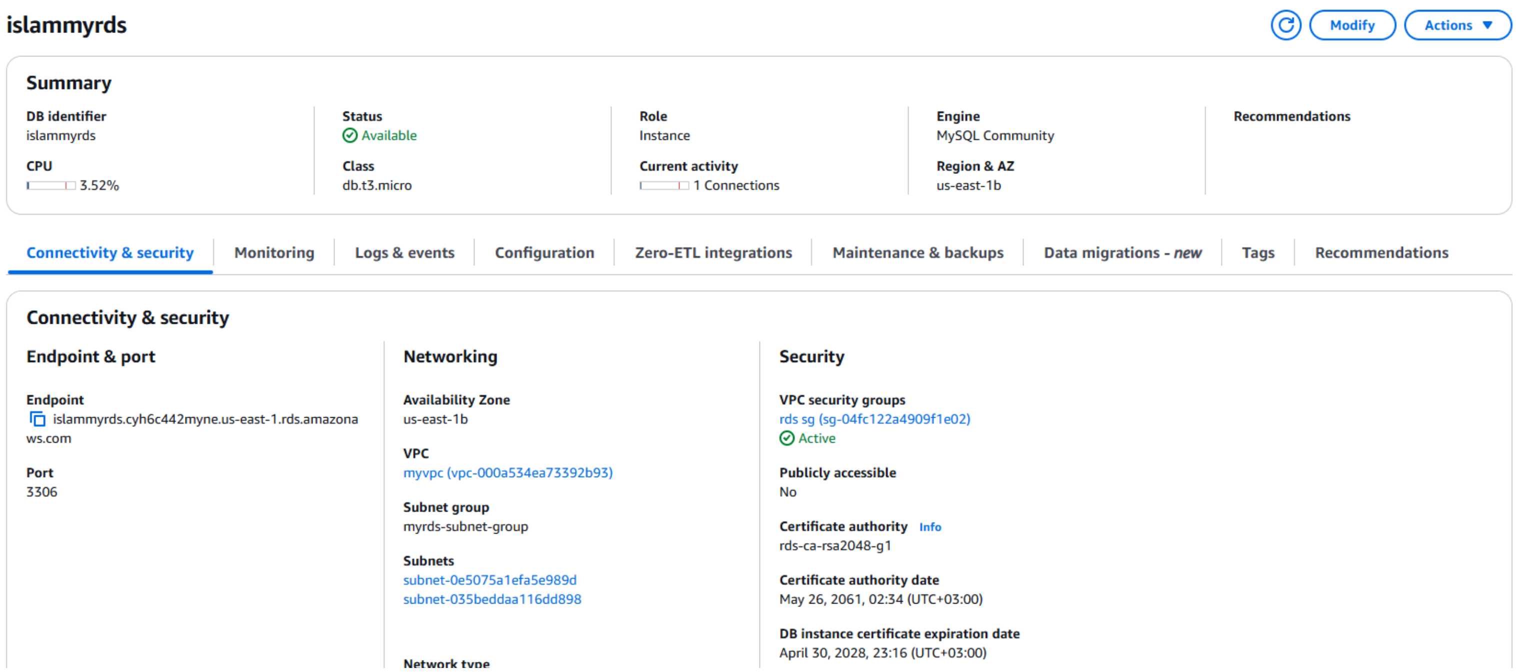
1-The code creates an RDS **MySQL** Database named `islammyrds` using **MySQL 5.7** and stores its data in a **20 GB** size with the **`gp2`** storage type.

2-t uses a small instance type **`db.t3.micro`** and is linked to the security group **`rds-sg`** to control access.

3-It specifies the RDS running location at `**us-east-1b**`

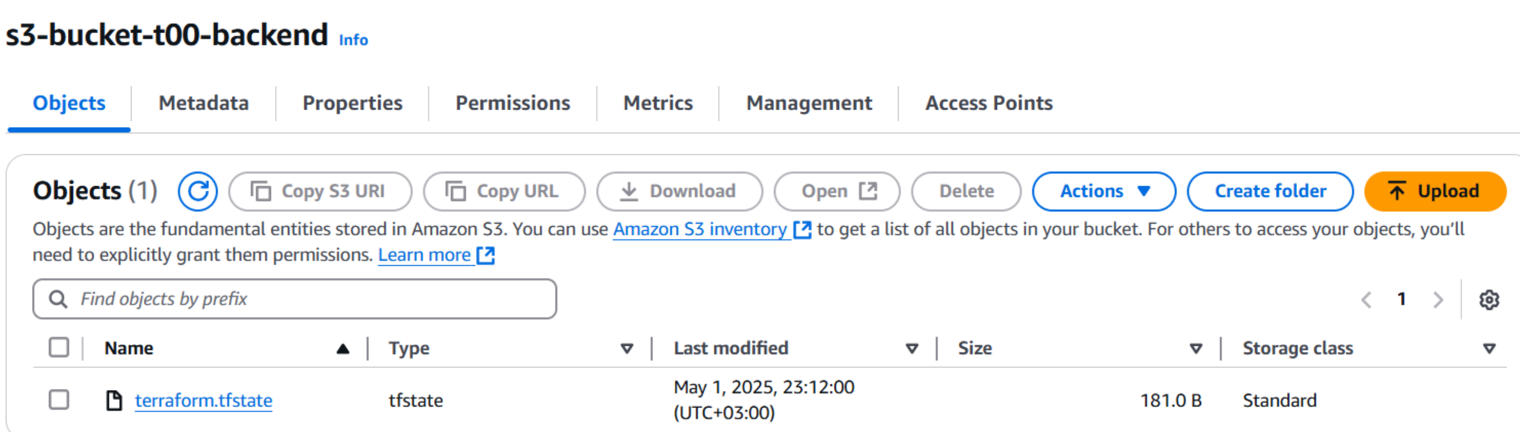
4-It relies on a subnet group called `**myrds-subnet-group**`, which includes the public and private subnets to distribute the network.

5-It determines the username and password for the database from Terraform variables (**`var.MYSQL_USER`** and **`var.MYSQL_PASSWORD`**) and uses `default.mysql5.7` as the parameter group.



Step 6: S3 (s3.tf)

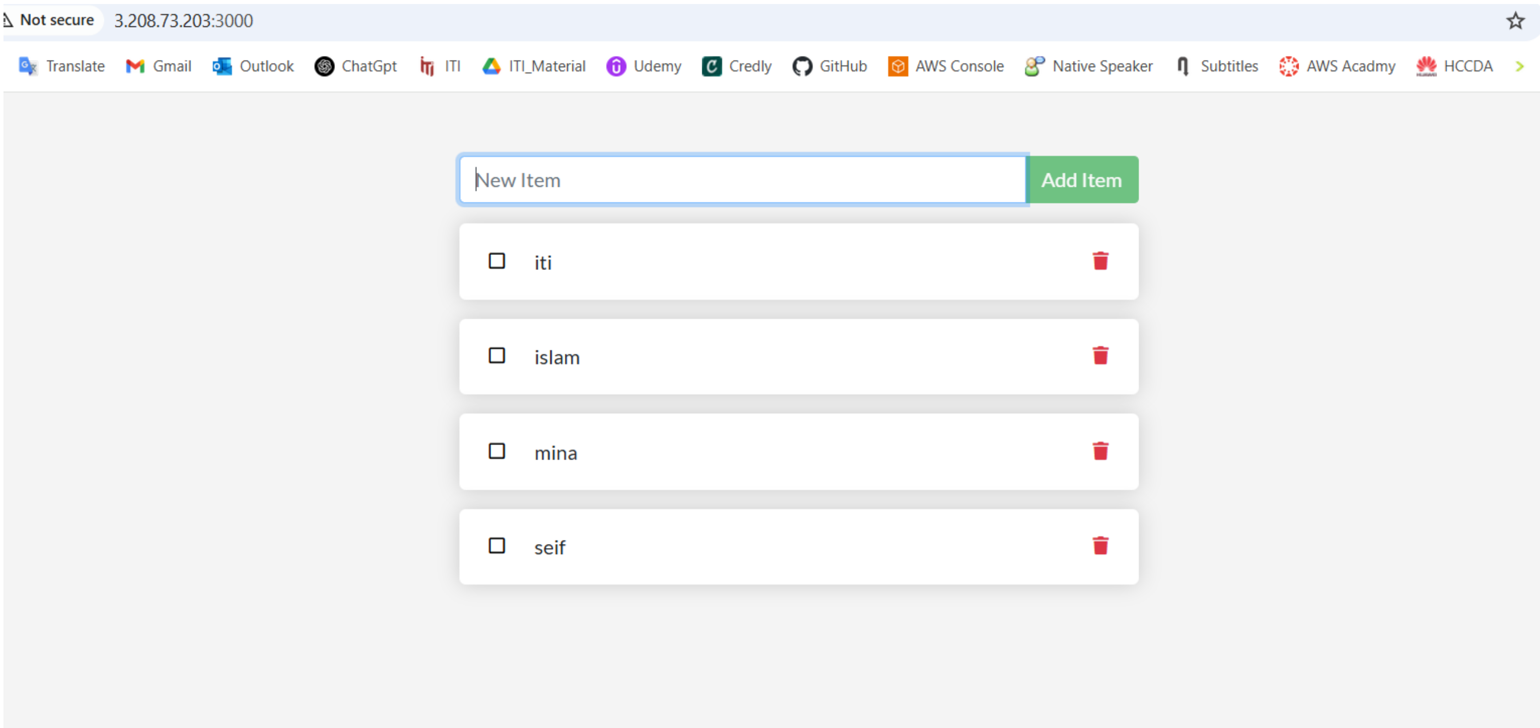
The code creates an S3 bucket named `**s3-bucket-t000-backend**` to be used as a backend for storing state. It adds tags to define the name (`My bucket`) and environment (`Dev`).



Step 7: Backend (backend.tf)

A Terraform backend stores state files in an S3 bucket named `**s3-bucket-t000-backend**` with a key of `**terraform.tfstate**`. This makes them available to the entire team for collaborative work.

Final result



☺ Thank You ☺